Model Assessment

On this page
Comparing Models
Probability Metrics
Classification Metrics
Optimal Cut-off

Comparing Models

One of the common concerns and questions in any model development is determining how "good" a model is or how well it performs. There are many different factors that determine this and most of them depend on the goals for the model. There are typically two different purposes for modeling - estimation and prediction. Estimation quantifies the expected change in our target variable associated with some relationship or change in the predictor variables. Prediction on the other hand is focused on predicting new target observations. However, these goals are rarely seen in isolation as most people desire a blend of these goals for their models. This section will cover many of the popular metrics for model assessment.

The first thing to remember about model assessment is that a model is only "good" in context with another model. All of these model metrics are truly model comparisons. Is an accuracy of 80% good? Depends! If the previous model used has an accuracy of 90%, then no the new model is not good. However, if the previous model used has an accuracy of 70%, then yes the model is good. Although we will show many of the calculations, at no place will we say that you must meet a certain threshold for your models to be considered "good" because these metrics are designed for comparison.

Some common model metrics are based on deviance/likelihood calculations. Three common logistic regression metrics based on these are the following:

1. AIC
2. BIC
3. Generalized (Nagelkerke) R^2

Without going into too much mathematical detail, the AIC is a crude, large sample approximation of leave-one-out cross validation. The BIC on the other hand favors a smaller model than the AIC as it penalizes model complexity more. In both AIC and BIC, lower values are better. However, there is no amount of lower that is better enough. Neither the AIC or BIC is necessarily better than the other however they may not always agree on the "best" model.

There are a number of "pseudo"-R^2 metrics for logistic regression. Here, higher values indicate a better model. The Generalized (Nagelkerke) R^2 is a metric to measure how much better a model (in terms of likelihood) is compared to the intercept only model. Therefore, we compare two models with this to see which one is "more better" than the intercept compared to the other. Essentially, they are both compared to the same baseline so whichever beats that baseline by more is the model we want. Even though it is bounded between 0 and 1, there is no interpretation to this metric like we had in linear regression.

We will be using the Ames, Iowa housing data set for this section. Let's see how we get these metrics in each of our softwares!

R Python SAS

R provides a lot of these metrics by default in the output of the `summary` function on our logistic regression objects. However, we can call them separately through the `AIC`, `BIC`, and `PseudoR2` functions as well. Note the `which = Nagelkerke` option in the `PseudoR2` function to get the correct R^2 value.

▼ Code

```
logit.model <- glm(Bonus ~ Gr_Liv_Area + factor(House_Style) + f
                             Fireplaces + factor(Full_Bath) + Lot_
                             TotRms_AbvGrd + Gr_Liv_Area:Fireplac
                    data = train, family = binomial(link = "logi

AIC(logit.model)
```

```
[1] 1287.964
```

▼ Code

```
BIC(logit.model)
```

```
[1] 1394.86
```

▼ Code

```
library(DescTools)

PseudoR2(logit.model, which = "Nagelkerke")
```

```
Nagelkerke
 0.7075796
```

We can see from the table above what the Generalized R^2 is for this model. Unlike SAS, this value has already been max-rescaled to be between 0 and 1.

Probability Metrics

Logistic regression is a model for predicting the **probability of an event**, not the occurrence of an event. Logistic regression **can** be used for classification as well. Good models should reflect both good metrics on probability and classification, but the importance of one over the other depends on the problem.

In this section we will focus on the probability metrics. Since we are predicting the probability of an event, we want our model to assign higher predicted probabilities to events and lower predicted probabilities to non-events.

Coefficient of Discrimination

The **coefficient of discrimination** (or discrimination slope) is the difference in average predicted probabilities of the actual events and non-events.

$$D = \bar{\hat{p}}_1 - \bar{\hat{p}}_0$$

The bigger this difference, this better our model does as separating the events and non-events through the predicted probabilities. We can also examine the histograms of these predictions for comparison as well.

Let's see how to calculate this in each of our softwares!

R Python SAS

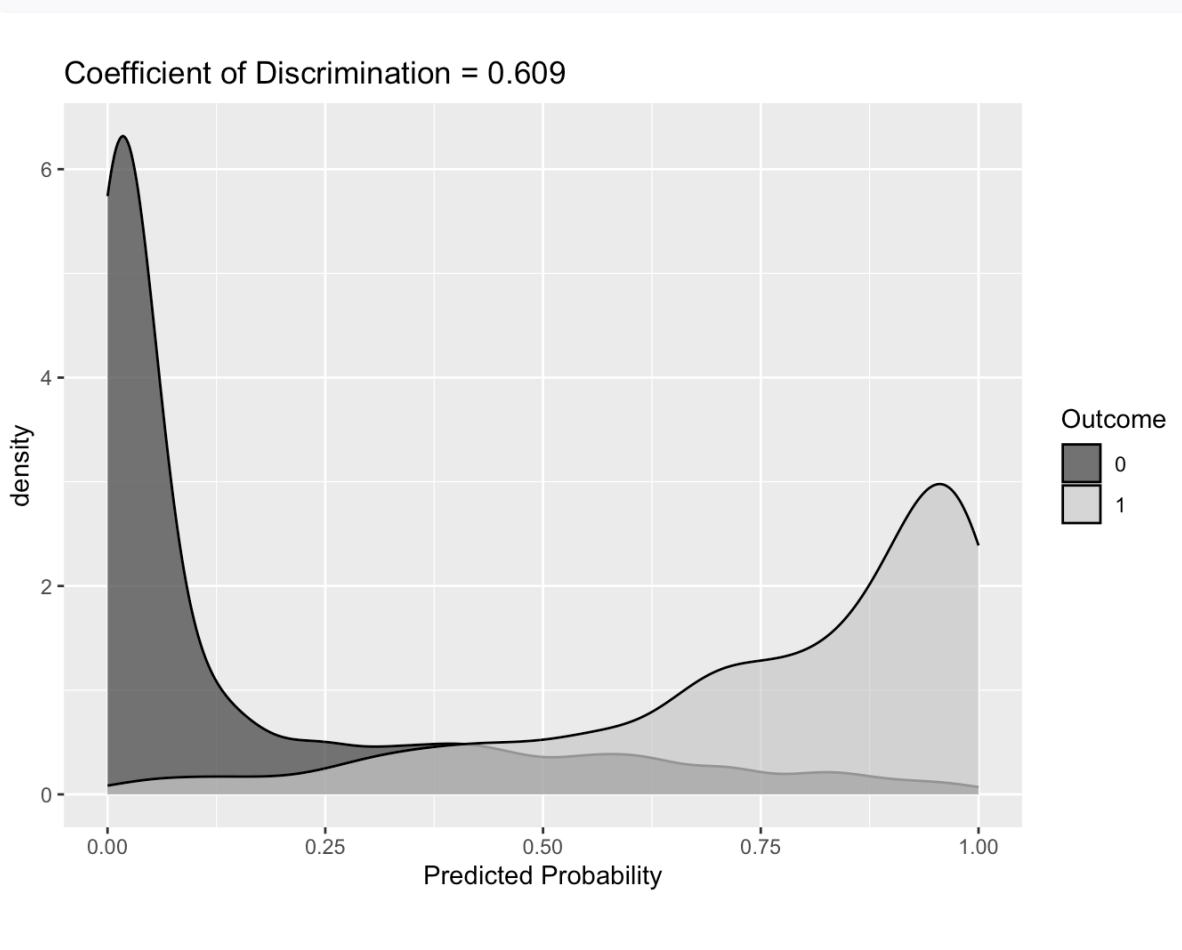
It is easy to calculate the difference of the average of two vectors in R which is exactly what we are doing. First, we use the `predict` function to gather the predictions from our model (`type = "response"` to get probabilities) and put them as a new column *p_hat* in our data frame. From there we just isolate out the events and non-events and save them in two vectors *p1* and *p0*. This is done by conditioning the data frame to only have rows where either `Bonus == 1` or `Bonus == 0`. Lastly, the coefficient of discrimination is just the difference of the mean of these two vectors. A plot of the two histograms overlaid on each other is also provided for visual inspection.

▼ Code

```
train$p_hat <- predict(logit.model, type = "response")

p1 <- train$p_hat[train$Bonus == 1]
p0 <- train$p_hat[train$Bonus == 0]
coef_discrim <- mean(p1) - mean(p0)

ggplot(train, aes(p_hat, fill = factor(Bonus))) +
  geom_density(alpha = 0.7) +
  scale_fill_grey() +
  labs(x = "Predicted Probability",
       fill = "Outcome",
       title = paste("Coefficient of Discrimination = ",
                     round(coef_discrim, 3), sep = ""))
```



The coefficient of discrimination is the difference of the average which is reported on the top of the plot above, 0.609. We can also see the two histograms of our predictions. These histograms don't seem to have too much overlap. There is a large group of 0's with low probabilities and a medium sized group of 1's with high probabilities. There is a little overlap where the model wasn't able to predict those observations as well.

Rank-Order Statistics

Rank-order statistics measure how well a model orders the predicted probabilities. Three common metrics that summarize things together are concordance, discordance, and ties. In these metrics every single combination of an event and non-event are compared against each other (1 event vs. 1 non-event). A concordant pair is a pair with the event having a higher predicted probability than the non-event - the model got the rank correct. A discordant pair is a pair with the event having a lower predicted probability than the non-event - the model got the rank wrong. A tied pair is a pair where the event and non-event have the same predicted probability - the model isn't sure how to distinguish between them. Models with higher concordance are considered better. The interpretation on concordance is that for all possible event and non-event combinations, the model assigned the higher predicted probability to the observation with the event *concordance%* of the time.

There are a host of other metrics that are based on these rank-statistics such as the c-statistic, Somer's D, and Kendall's τ_α. The calculations for these are as follows:

$$c = Concordance + 1/2 \times Tied$$

$$D_{xy} = 2c - 1$$

$$\tau_\alpha = \frac{Condorant - discordant}{0.5 * n * (n-1)}$$

With all of these, higher values of concordant pairs result in higher values of these metrics.

Let's see how to calculate these in each of our softwares!

R Python SAS

Although not provided immediately in the model summary, R can easily provide a couple of the metrics above with the and `somers2` function.

▼ Code

```
library(Hmisc)

somers2(train$p_hat, train$Bonus)
```

```
        C          Dxy            n      Missing 
0.9428394    0.8856789 2051.0000000    0.0000000 
```

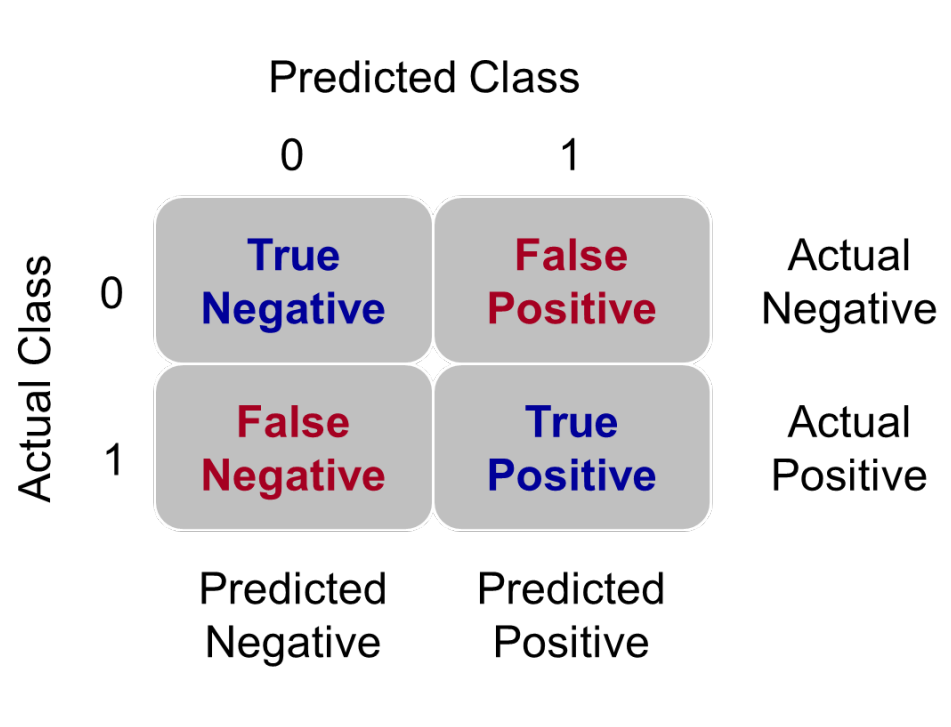
As we can see from the output, our model assigned the higher predicted probability to the observation with the bonus eligible home 94.3% of the time (the `C` in the output). Somer's D had a value of 0.885 (the `Dxy` in the output). Just like with other model metrics, we cannot say whether these are "good" values of these metrics as they are meant for comparison. If these values are higher than the same values from another model, then this model is better than the other model.

Classification Metrics

Logistic regression is a model for predicting the **probability of an event**, not the occurrence of an event. Logistic regression **can** be used for classification as well. Good models should reflect both good metrics on probability and classification, but the importance of one over the other depends on the problem.

In this section we will focus on the classification metrics. We want a model to correctly classify events and non-events. Classification forces the model to predict either an event or non-event for each observation based on the predicted probability for that observation. For example, $\hat{y}_1 = 1$ if $\hat{p}_i > 0.5$. These are called cut-offs or thresholds. However, strict classification-based measures completely discard any information about the actual quality of the model's predicted probabilities.

Many of the metrics around classification try to balance different pieces of the **classification table** (also called the **confusion matrix**). An example of one is shown below.

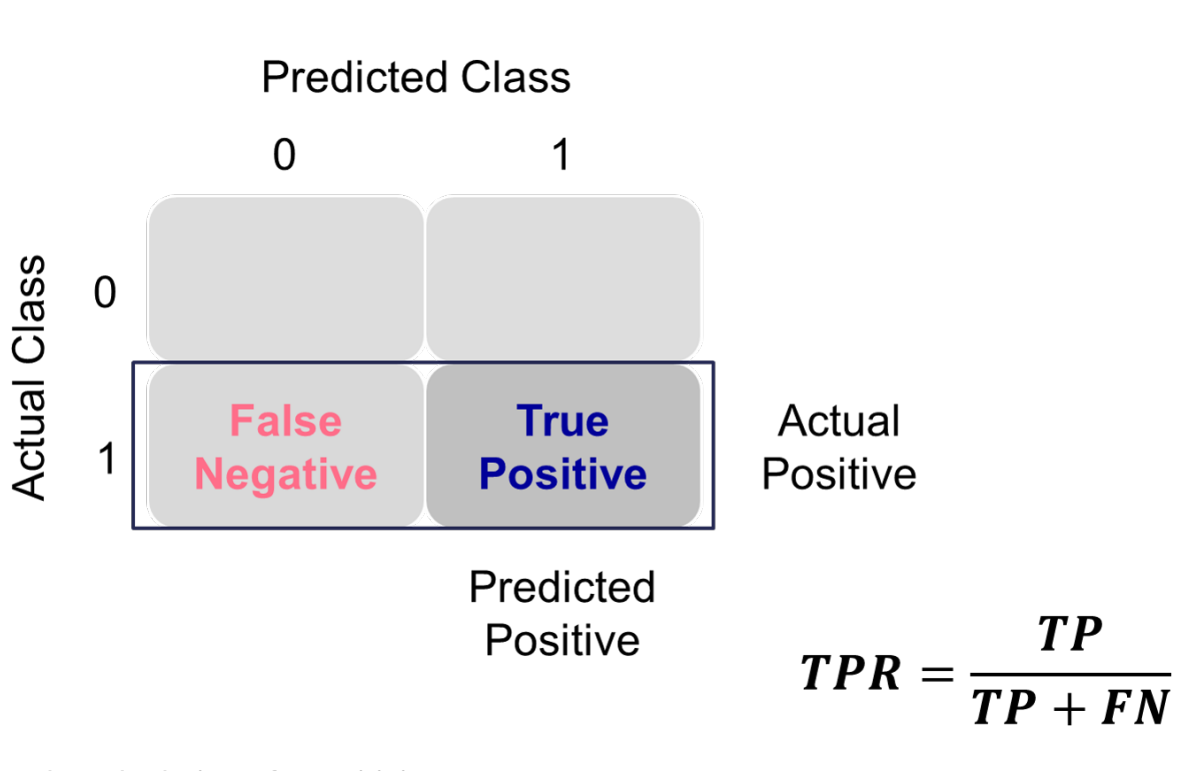


Classification Table Example

Let's examine the different pieces of the classification table that people jointly focus on.

Sensitivity & Specificity

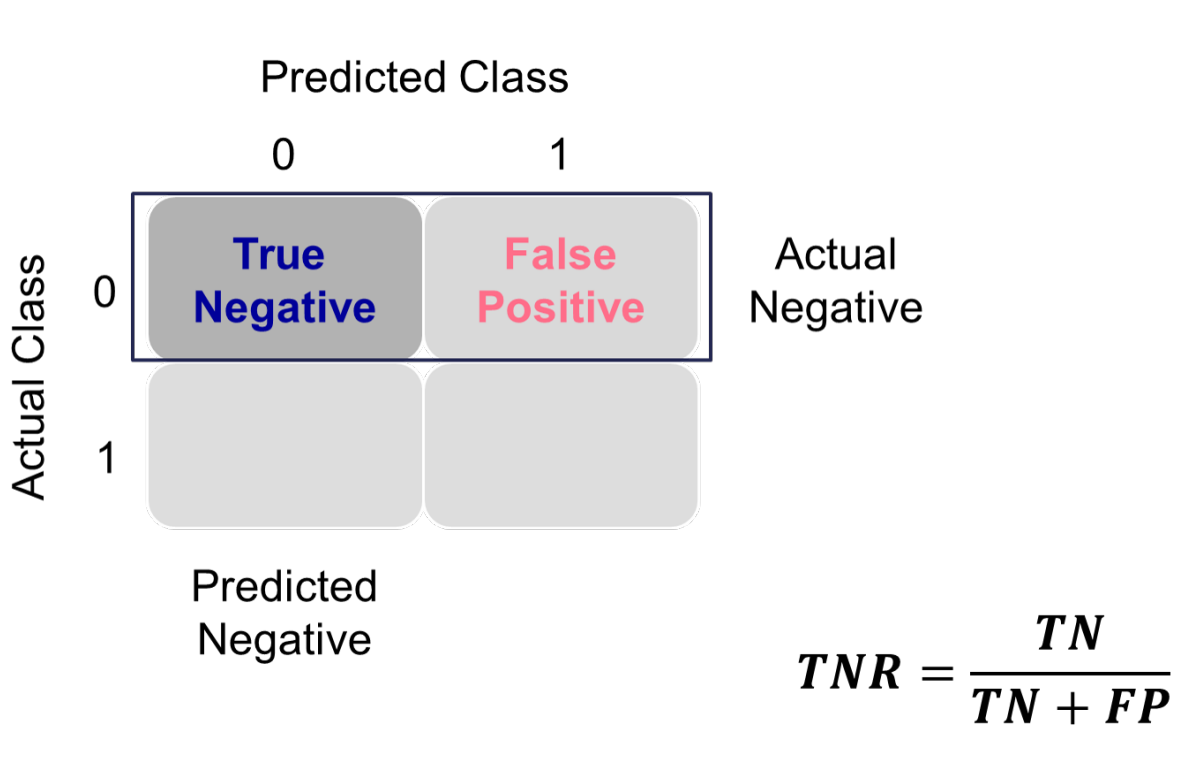
Sensitivity is the proportion of times you were able to predict an event in the groups of actual events. Of the actual events, the proportion of the time you correctly predicted an event. This is also called the **true positive rate**. This is also just another name for recall.



$$TPR = \frac{TP}{TP + FN}$$

Example Calculation of Sensitivity

This is balanced typically with the specificity. Specificity is the proportion of times you were able to predict a non-event in the group of actual non-events. Of the actual non-events, the proportion of the time you correctly predicted non-event. This is also called the **true negative rate**.



$$TNR = \frac{TN}{TN + FP}$$

Example Calculation of Specificity

These offset each other in a model. One could easily maximize one of these at the cost of the other. To get maximum sensitivity you can just predict every observations is an event, however this would drop your specificity to 0. The reverse is also true. Those who focus on sensitivity and specificity want balance in each. One measure for the optimal cut-off from a model is the **Youden's Index** (or Youden's J Statistic). This is easily calculated as $J = sensitivity + specificity - 1$. The optimal cut-off for determining predicted events and non-events would be at the point where this is maximized.

Let's see how to do this in each of our softwares!

R Python SAS

R produces classification tables for any cut-off that you determine. Remember that a cut-off is simply the point where you decide to predict an event and non-event from your predicted probabilities. The `table` function creates the classification table for us after using the `ifelse` function to define our cut-off at 0.5.

▼ Code

```
train <- train %>%
  mutate(Bonus_hat = ifelse(p_hat > 0.5, 1, 0))

table(train$Bonus_hat, train$Bonus)
```

```
       0    1
  0 1062  127
  1  149  713
```

We want to look at all classification tables for all values of cut-offs between 0 and 1. We can easily loop through this calculation with a `for` loop in R. However, the `measureit` function can do this for us. The inputs for this function are the predicted probabilities first, followed by the target variable. The `measure` option allows you to define additional measures to calculate at each cut-off. In the code below we ask for accuracy (`ACC`), sensitivity (`SENS`), and specificity (`SPEC`). From there we combine these variables into a single dataframe and print the observations with the `head` function.

▼ Code

```
library(ROCit)

logit_meas <- measureit(train$p_hat, train$Bonus, measure = c("
summary(logit_meas)
```

```
       Length Class  Mode   
Cutoff 2031   -none- numeric
Depth  2031   -none- numeric
TP     2031   -none- numeric
FP     2031   -none- numeric
TN     2031   -none- numeric
FN     2031   -none- numeric
ACC    2031   -none- numeric
SENS   2031   -none- numeric
SPEC   2031   -none- numeric
```

▼ Code

```
youden_table <- data.frame(Cutoff = logit_meas$Cutoff, Sens = l
head(youden_table, n = 10)
```

```
      Cutoff        Sens      Spec
1        Inf 0.000000000 1.0000000
2  0.9999996 0.000000000 0.9991742
3  0.9999963 0.001190476 0.9991742
4  0.9999952 0.002380952 0.9991742
5  0.9999786 0.003571429 0.9991742
6  0.9999653 0.004761905 0.9991742
7  0.9999573 0.005952381 0.9991742
8  0.9999546 0.007142857 0.9991742
9  0.9999381 0.008333333 0.9991742
10 0.9998792 0.009523810 0.9991742
```

We could calculate the Youden Index by hand and then rank the new dataframe by this value, however, `rocit` function will gives this to us automatically in the next piece of code.

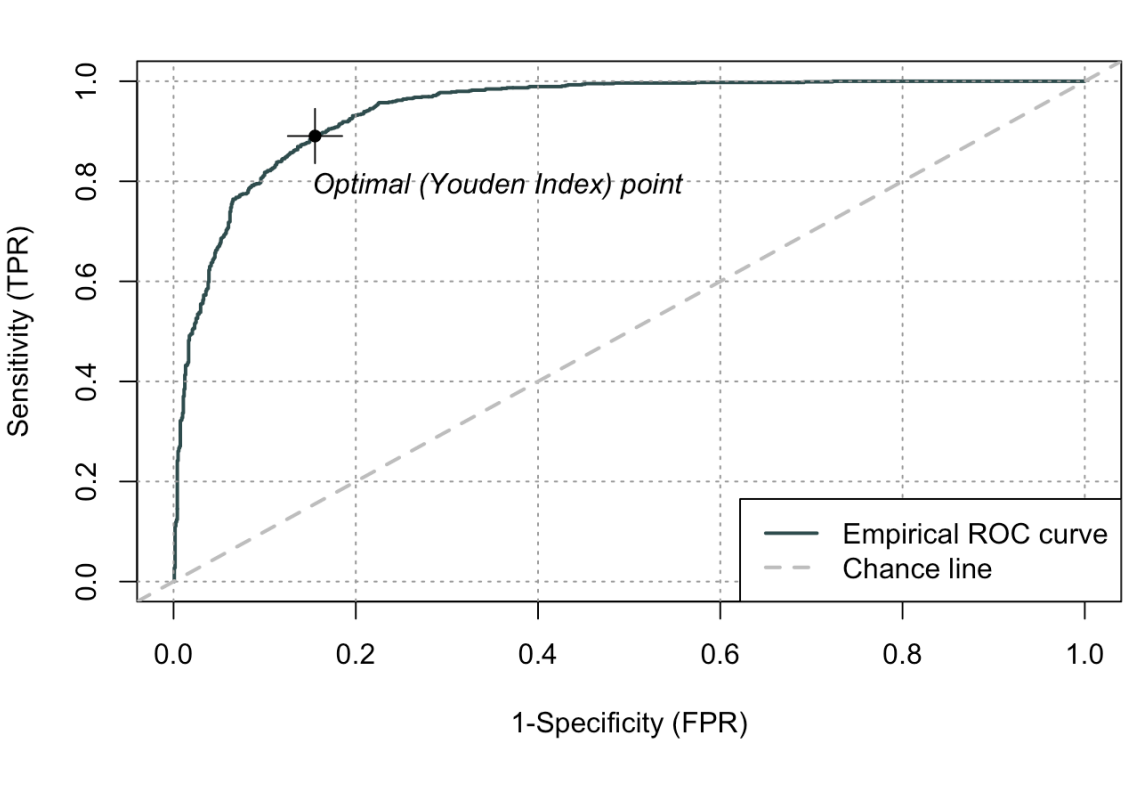
Another commonly used visual for the balance of sensitivity and specificity across all of the cut-offs is the Receiver Operator Characteristic curve. Commonly known as the ROC curve. The ROC curve plots the balance of sensitivity vs. specificity. The "best" ROC curve is the one that reaches to the upper left hand side of the plot as that would imply that our model has both high levels of sensitivity and specificity. The worst ROC curve is represented by the diagonal line in the plot since that would imply our model is as good as randomly assigning events and non-events to our observations. This leads some to calculate the area under the ROC curve (typically called AUC) as a metric summarizing the curve itself. The math won't be shown here, but the AUC is equal to the c-statistic in the Rank-order statistics section. Isn't math fun!?!?

R easily produces ROC curves from a variety of functions. A popular, new function is the `rocit` function. Using the `plot` function on the `rocit` object gives the ROC curve. By calling the `$optimal` element of the plot of the `rocit` object, it gives the value of the Youden's Index (`value` in the output) along with the respective cut-off that corresponds to that maximum Youden value. The `summary` function on the `rocit` object will report the AUC value. We can also get confidence intervals around our AUC values (`ciAUC` function) and ROC curves (`ciROC` function).

▼ Code

```
logit_roc <- rocit(train$p_hat, train$Bonus)
plot(logit_roc)

plot(logit_roc)$optimal
```



```
    value       FPR       TPR    cutoff 
0.7352326 0.1552436 0.8904762 0.4229724 
```

▼ Code

```
summary(logit_roc)
```

```
 Method used: empirical     
 Number of positive(s): 840 
 Number of negative(s): 1211
 Area under curve: 0.9428   
```

▼ Code

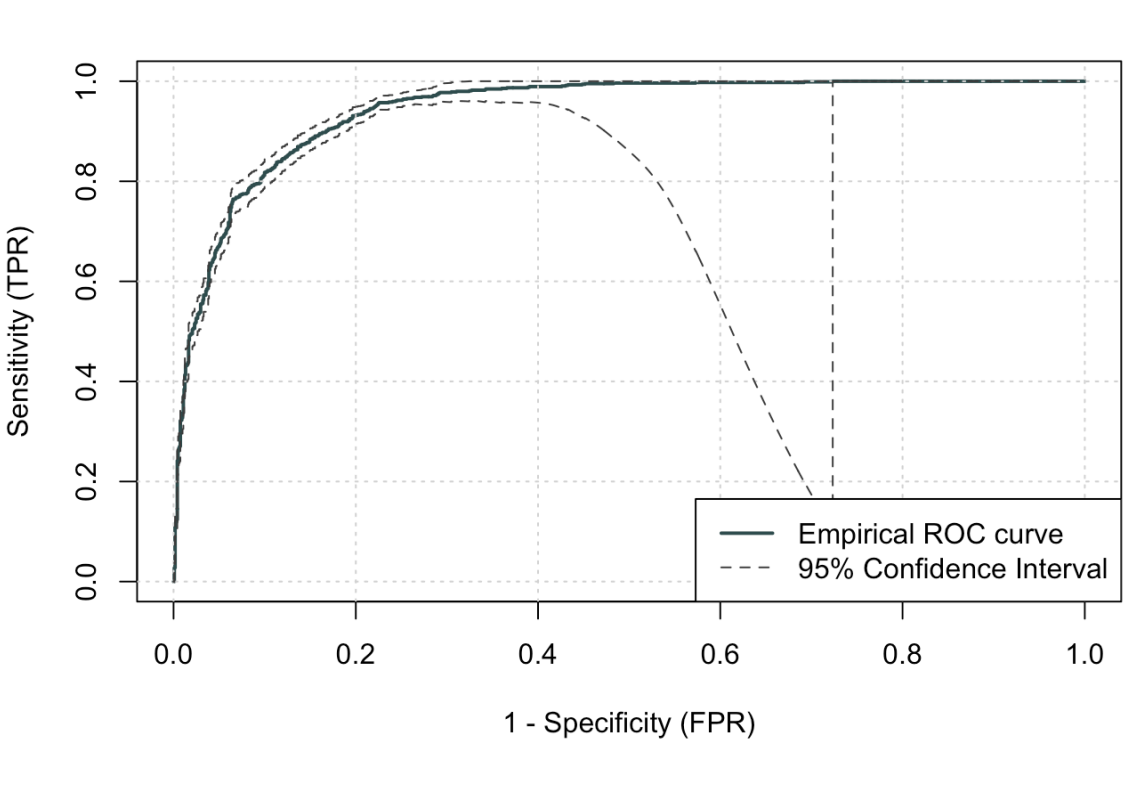
```
ciAUC(logit_roc, level = 0.99)
```

```
   estimated AUC : 0.942839447917884 
   AUC estimation method : empirical 
   
   CI of AUC
   confidence level = 99%
   lower = 0.927916041014629     upper = 0.957762854821139 
```

▼ Code

```
plot(ciROC(logit_roc))
```

```
Warning in regularize.values(x, y, ties, missing(ties), na.rm =
collapsing to unique 'x' values
```



We can see that the highest Youden J statistic had a value of 0.7352. This took place at a cut-off of 0.423. Therefore, according to the Youden Index at least, the optimal cut-off for our model is 0.423. In other words, if our model predicts a probability above 0.423 then we should call this an event. Any predicted probability below 0.423 should be called a non-event. We can also see that the area under our ROC curve is 0.9428. Similar to other metrics, we cannot say whether this is a "good" value of AUC, only if it is better or worse than another model's AUC.

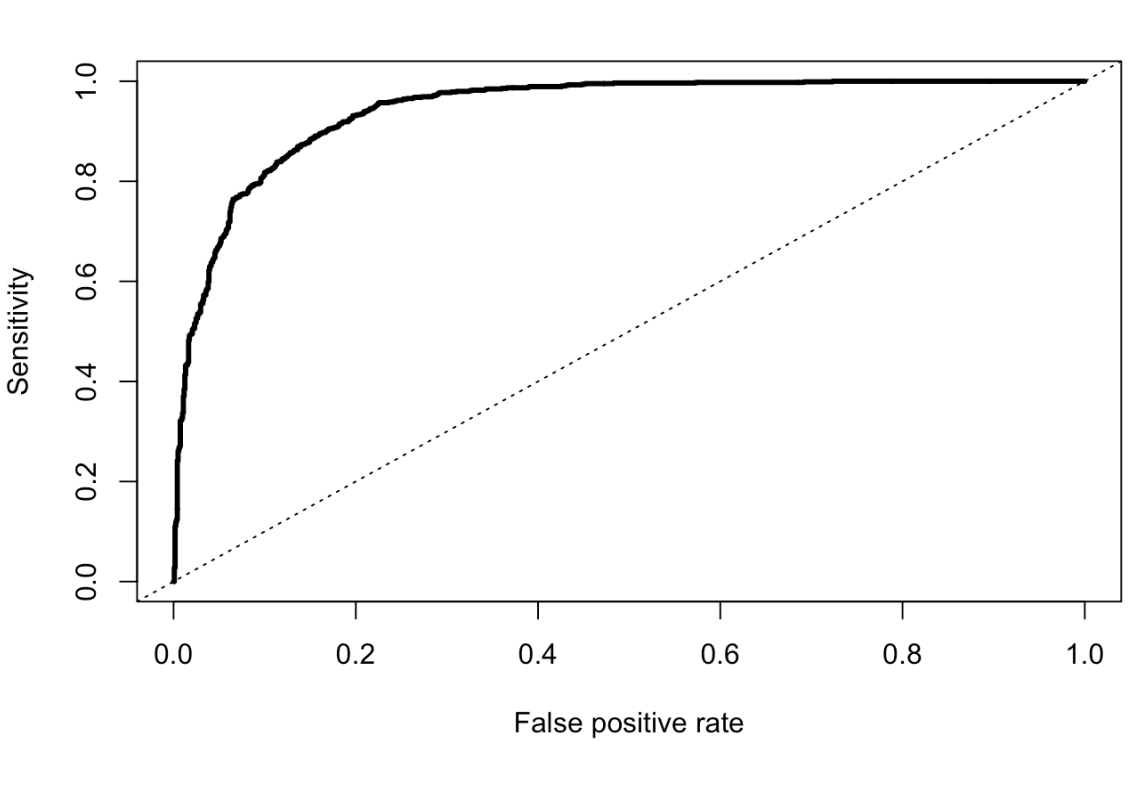
Another common function is the `performance` function that produces many more plots than the ROC curve. Here the ROC curve is obtained by plotting the true positive rate by the false positive rate using the `measure = "tpr"` and `"x.measure = "fpr"` options. The AUC is also obtained from the `performance` function by calling the `measure = "auc"` option.

▼ Code

```
library(ROCR)

pred <- prediction(train$p_hat, factor(train$Bonus))

perf <- performance(pred, measure = "sens", x.measure = "fpr")
plot(perf, lwd = 3, colorize = FALSE, colorkey = FALSE)
abline(a = 0, b = 1, lty = 3)
```



▼ Code

```
performance(pred, measure = "auc")@y.values
```

```
[[1]]
[1] 0.9428394
```

K-S Statistic

One of the most popular metrics for classification models in the finance and banking industry is the KS statistic. The two sample KS statistic can determine if there is a difference between two cumulative distribution functions. The two cumulative distribution functions of interest to us are the predicted probability distribution functions for the event and non-event target group. The KS D statistic is the maximum distance between these two curves - calculated by the maximum difference between the true positive and false positive rates, $D = \max_{depth}(TPR - FPR) = \max_{depth}(Sensitivity + Specificity - 1)$. Notice, this is the same as maximizing the Youden Index.

The optimal cut-off for determining predicted events and non-events would be at the point where this D statistic (Youden Index) is maximized.

Let's see how to do this in each of our softwares!

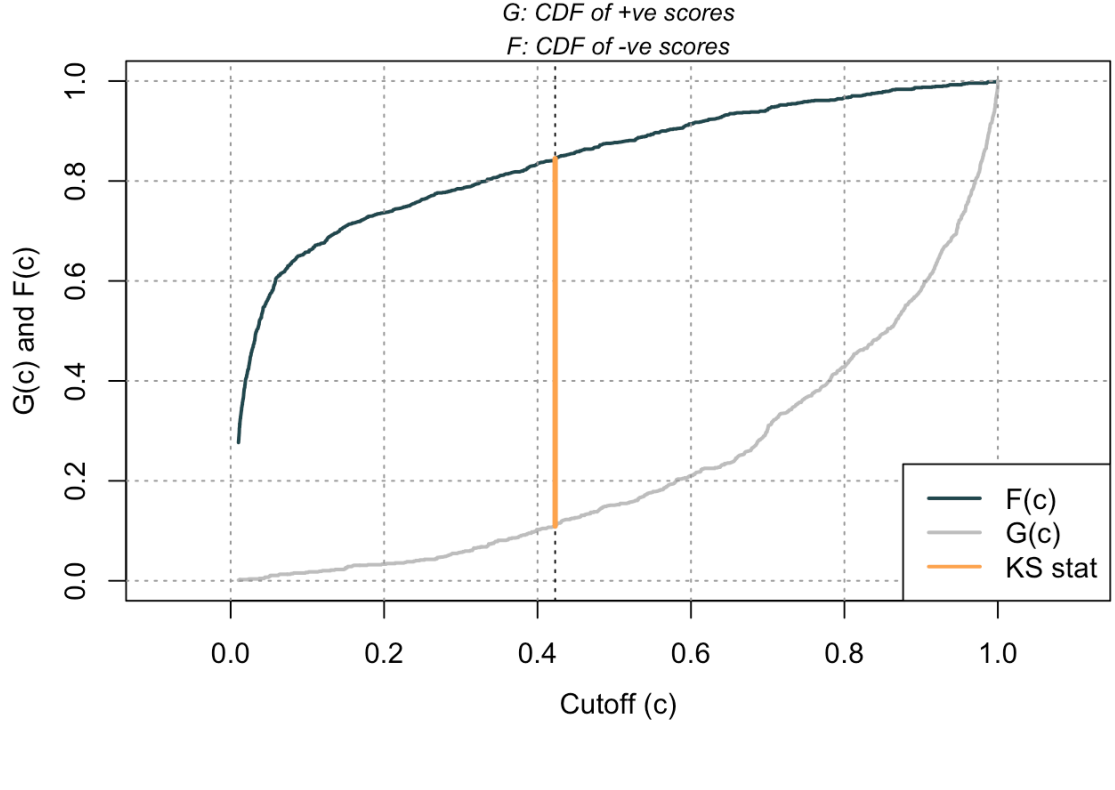
R Python SAS

Using the same `rocit` object from the section on sensitivity and specificity (here called `logit_roc`) we can also calculate the KS statistic and plot the two cumulative distribution functions it represents. The `ksplot` function will plot the two cumulative distribution functions as well as highlight the cut-off (or threshold) where they are most separated. This point corresponds the D statistic mentioned above as well as the Youden's Index. By calling the `KS stat` and `KS Cutoff` elements from this KS plot, we can get the optimal cut-off as well as the value at this cut-off.

▼ Code

```
ksplot(logit_roc)

ksplot(logit_roc)$`KS stat`
```



```
[1] 0.7352326
```

▼ Code

```
ksplot(logit_roc)$`KS Cutoff`
```

```
[1] 0.4229724
```

As we saw in the previous section, the optimal cut-off according to the KS-statistic would be at 0.423. Therefore, according to the KS statistic at least, the optimal cut-off for our model is 0.423. In other words, if our model predicts a probability above 0.423 then we should call this an event. Any predicted probability below 0.423 should be called a non-event. The KS D statistic is reported as 0.7352 which is equal to the Youden's Index value.

Another way to calculate this is by hand using the `performance` function we saw in the previous section as well. Using the `measure = "tpr"` and `x.measure = "fpr"` options, we can calculate the true positive rate and false positive rate across all our predictions. From there we can just use the `max` function to calculate the value of maximum difference between the two - the KS statistic. Finding the cut-off at this point is a little trickier with some of the needed R functions, but we essentially search the for alpha values (here the cut-offs) for the point where the KS statistic is maximized.

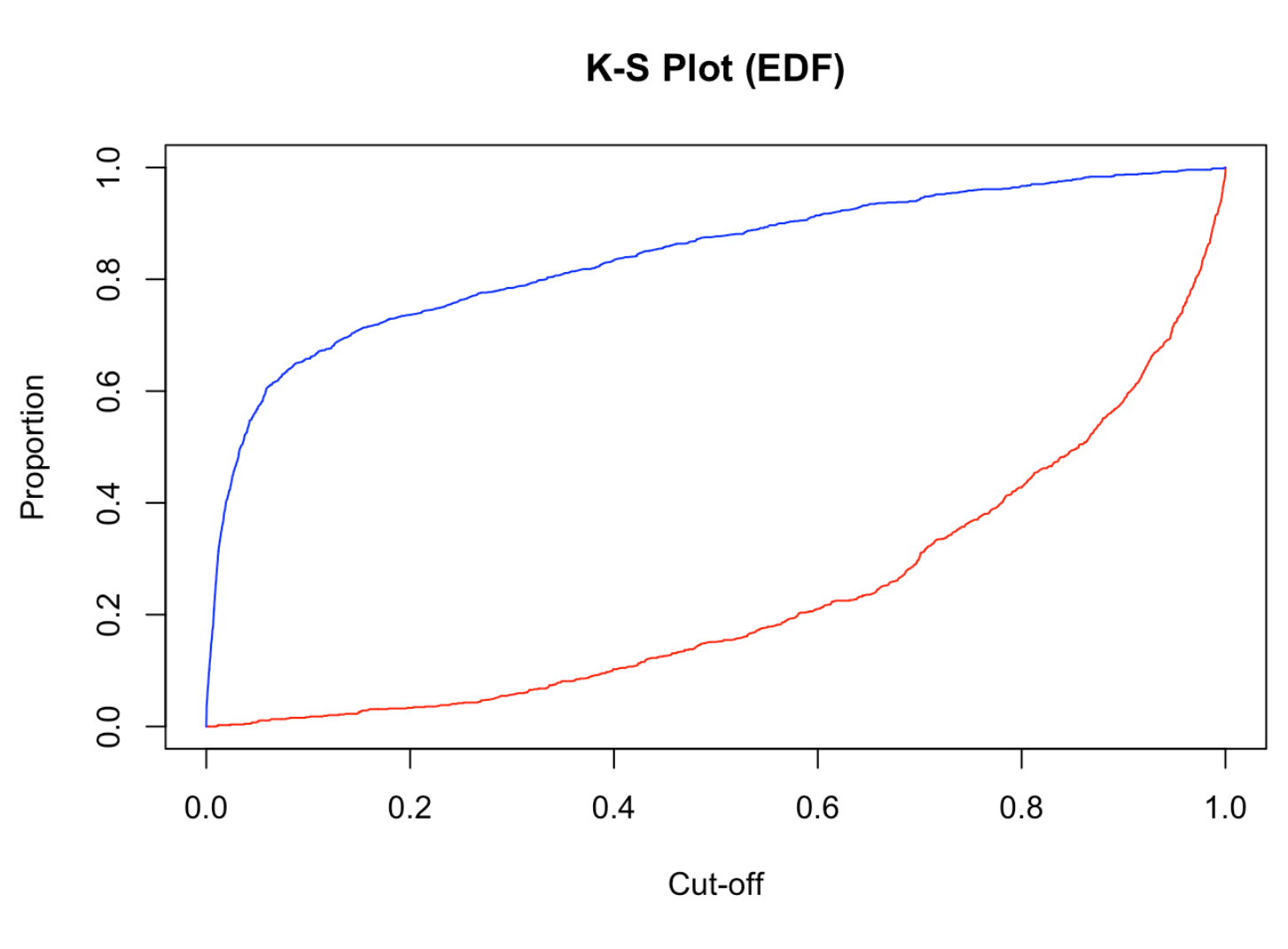
▼ Code

```
perf <- performance(pred, measure = "tpr", x.measure = "fpr")
KS <- max(perf@y.values[[1]] - perf@x.values[[1]])
cutoffAtKS <- unlist(perf@alpha.values)[which.max(perf@y.values
print(c(KS, cutoffAtKS))
```

```
[1] 0.7352326 0.4229724
```

▼ Code

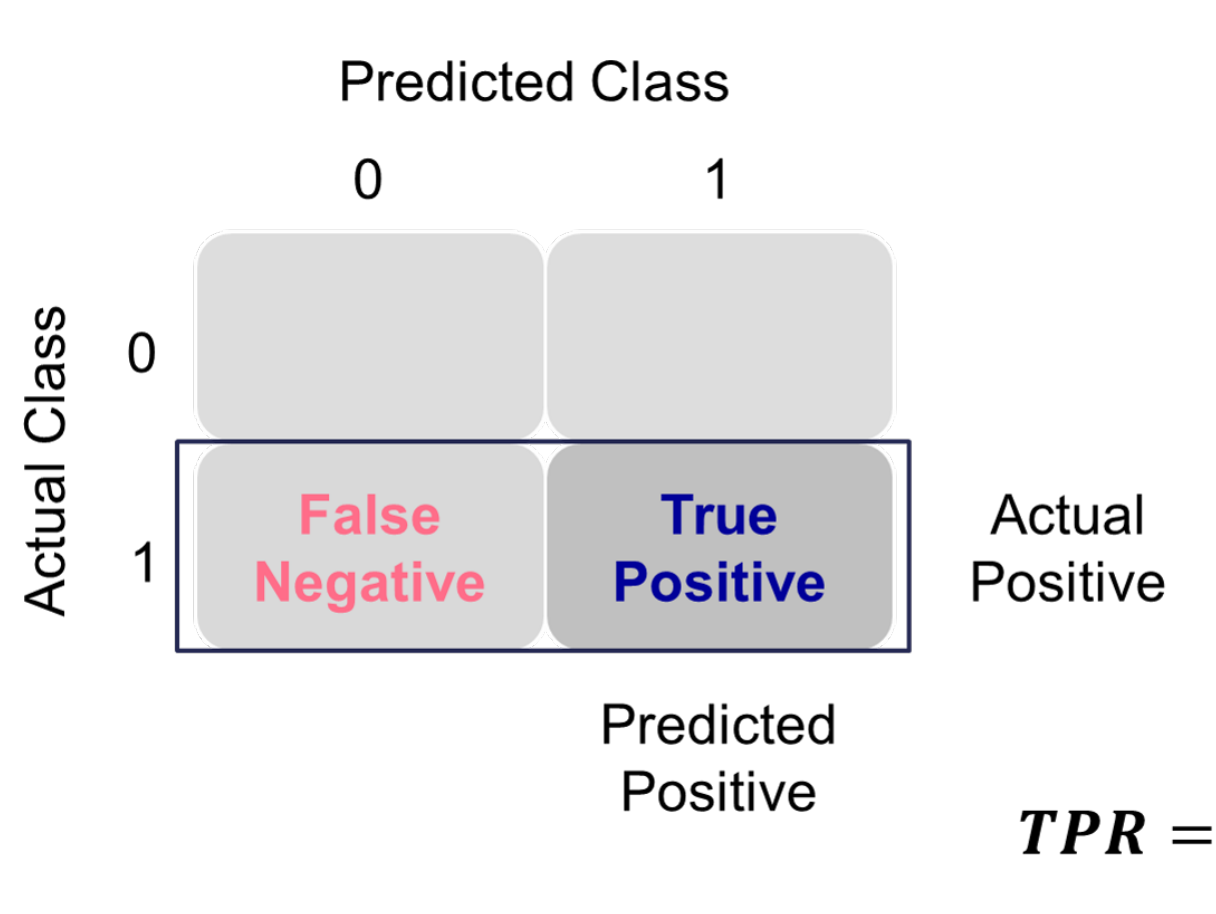
```
plot(x = unlist(perf@alpha.values), y = (1-unlist(perf@y.values
     type = "l", main = "K-S Plot (EDF)",
     xlab = 'Cut-off',
     ylab = "Proportion",
     col = "red")
lines(x = unlist(perf@alpha.values), y = (1-unlist(perf@x.value
```



From the output we can see the KS D statistic at 0.7352. The predicted probability that this occurs at (the optimal cut-off) is defined at 0.423 as we previously saw.

Precision & Recall

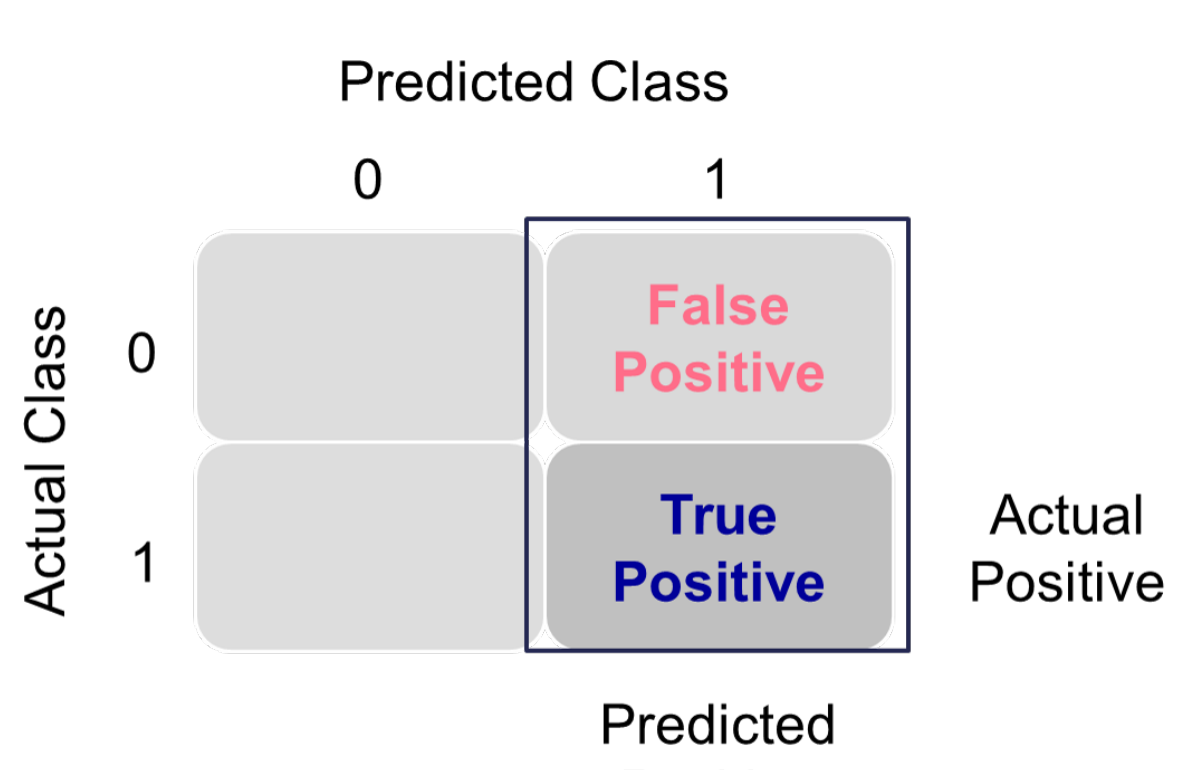
Precision and recall are another way to view a classification table from a model. Recall is the proportion of times you were able to predict an event in the groups of actual events. Of the actual events, the proportion of the time you correctly predicted an event. This is also called the **true positive rate**. This is also just another name for sensitivity.



$$TPR = \frac{TP}{TP + FN}$$

Example Recall Calculation

This is balanced here with the precision. Precision is the proportion predicted events that were actually events. Of the predicted events, the proportion of the time they actually were events. This is also called the **positive predictive value**. Precision is growing in popularity as a balance to recall/sensitivity as compared to specificity.



$$PPV = \frac{TP}{TP + FP}$$

Example Precision Calculation

These offset each other in a model. One could easily maximize one of these at the cost of the other. To get maximum recall you can just predict all events, however this would drop your precision. Those who focus on precision and recall want balance in each. One measure for the optimal cut-off from a model is the **F1 Score**. This is calculated as the following:

$$F_1 = 2 \times \frac{precision \times recall}{precision + recall}$$

The optimal cut-off for determining predicted events and non-events would be at the point where this is maximized.

Let's see how to do this in each of our softwares!

R Python SAS

R produces classification tables for any cut-off that you determine. Remember that a cut-off is simply the point where you decide to predict an event and non-event from your predicted probabilities.

We want to look at all classification tables for all values of cut-offs between 0 and 1. We can easily loop through this calculation with a `for` loop in R. However, the `measureit` function can do this for us. The inputs for this function are the predicted probabilities first, followed by the target variable. The `measure` option allows you to define additional measures to calculate at each cut-off. In the code below we ask for precision (`PREC`), sensitivity (`SENS`), and F1-score (`FSCR`). From there we combine these variables into a single dataframe and print the observations with the `print` function.

▼ Code

```
logit_meas <- measureit(train$p_hat, train$Bonus, measure = c("
summary(logit_meas)
```

```
       Length Class  Mode   
Cutoff 2031   -none- numeric
Depth  2031   -none- numeric
TP     2031   -none- numeric
FP     2031   -none- numeric
TN     2031   -none- numeric
FN     2031   -none- numeric
PREC   2031   -none- numeric
REC    2031   -none- numeric
FSCR   2031   -none- numeric
```

▼ Code

```
fscore_table <- data.frame(Cutoff = logit_meas$Cutoff, FScore =
head(arrange(fscore_table, desc(FScore)), n = 10)
```

```
      Cutoff    FScore
1  0.4229724 0.8423423
2  0.4113195 0.8421053
3  0.4293401 0.8419263
4  0.4225667 0.8418683
5  0.4254168 0.8418079
6  0.4306012 0.8417470
7  0.4235192 0.8416901
8  0.3918903 0.8416390
9  0.4549082 0.8415614
10 0.4029919 0.8415179
```

We can see that the highest F1 score had a value of 0.842. This took place at a cut-off of 0.423. Therefore, according to the F1 score at least, the optimal cut-off for our model is 0.423. In other words, if our model predicts a probability above 0.423 then we should call this an event. Any predicted probability below 0.423 should be called a non-event. This matches up with the Youden's Index from above. This is **not** always the case. We just got lucky in this example.

Another common calculation using the precision of a model is the model's **lift**. The lift of a model is simply calculated as the ratio of Precision to the population proportion of the event - $Lift = PPV/\pi_1$. The interpretation of lift is really nice for explanation. Let's imagine that your lift was 3 and your population proportion of events was 0.2. This means that in the top 20% of your customers, your model predicted 3 times the events as compared to you selecting people at random. Sometimes people plot lift charts where they plot the precision at all the different values of the population proportion (called depth).

Again, we can use the `rocit` object (called `logit_roc`) from earlier. The `gainstable` function breaks the data down into 10 groups (or buckets) ranked from highest predicted probability to lowest. We can use the `plot` function on this new object along with the `type` option to get a variety of useful plots. If you want more than 10 buckets for your data you can always use the `ngroup` option to specify how many you want.

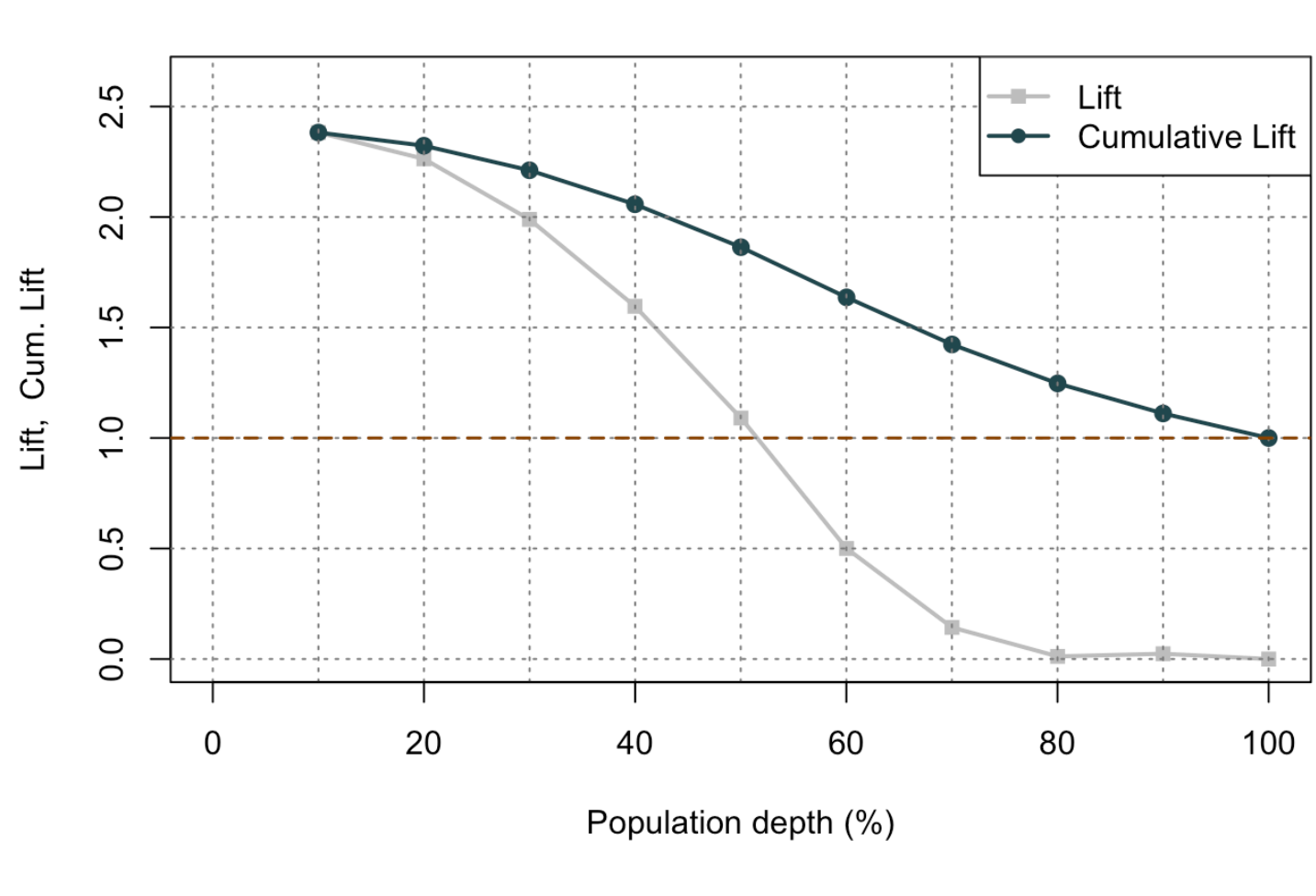
▼ Code

```
logit_lift <- gainstable(logit_roc)
print(logit_lift)
```

```
   Bucket Obs CObs Depth Resp CResp RespRate CRespRate CCapRate
1       1 205  205   0.1  200   200    0.976     0.976    0.238
2       2 205  410   0.2  190   390    0.927     0.951    0.464
3       3 205  615   0.3  167   557    0.815     0.906    0.663
4       4 205  820   0.4  134   691    0.654     0.843    0.823
5       5 206 1026   0.5   92   783    0.447     0.763    0.932
6       6 205 1231   0.6   42   825    0.205     0.670    0.982
7       7 205 1436   0.7   12   837    0.059     0.583    0.996
8       8 205 1641   0.8    1   838    0.005     0.511    0.998
9       9 205 1846   0.9    2   840    0.010     0.455    1.000
10     10 205 2051   1.0    0   840    0.000     0.410    1.000
```

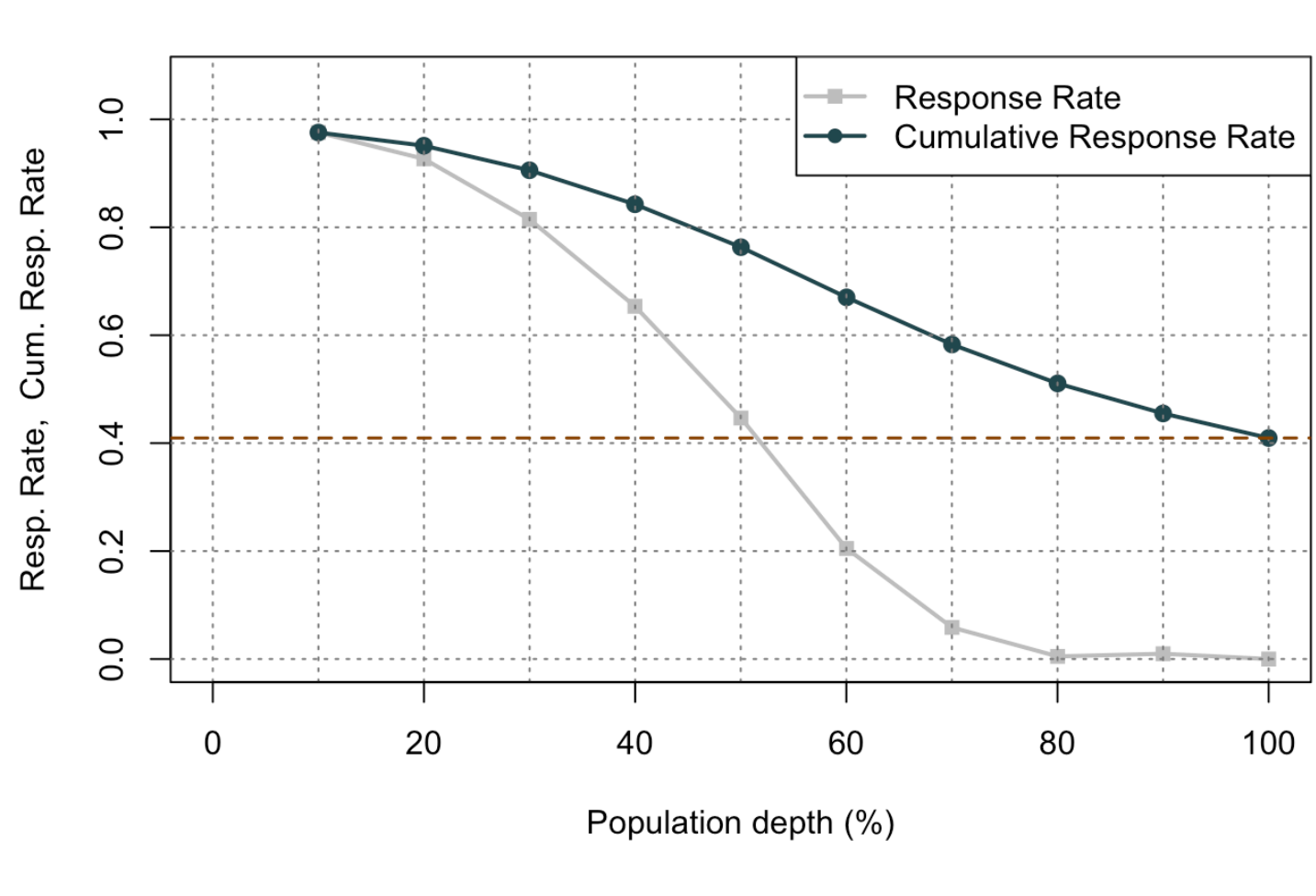
▼ Code

```
plot(logit_lift, type = 1)
```



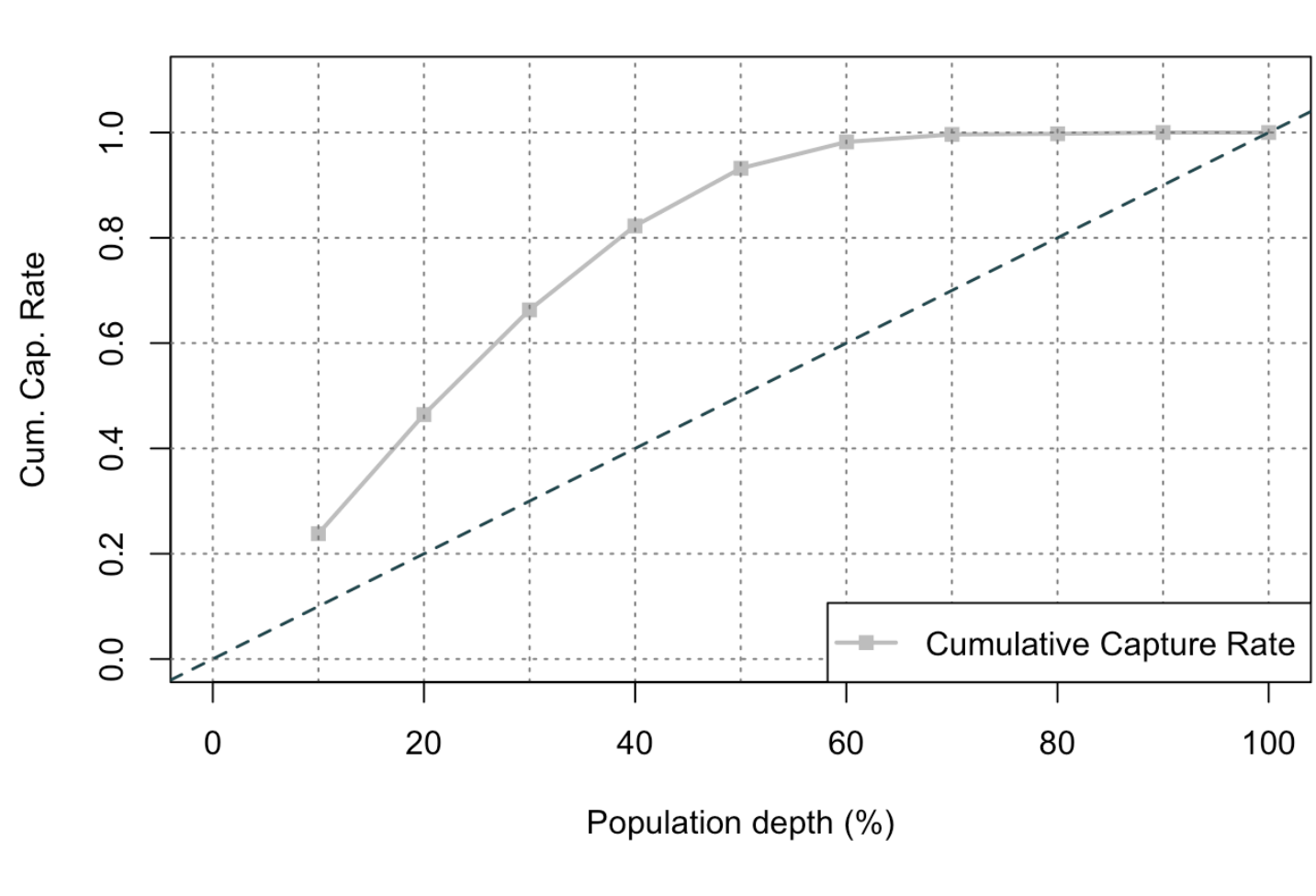
▼ Code

```
plot(logit_lift, type = 2)
```



▼ Code

```
plot(logit_lift, type = 3)
```



▼ Code

```
logit_lift <- gainstable(logit_roc, ngroup = 15)
print(logit_lift)
```

```
   Bucket Obs CObs Depth Resp CResp RespRate CRespRate CCapRate
1       1 137  137 0.067  132   132    0.964     0.964    0.157
2       2 136  273 0.133  132   264    0.971     0.967    0.314
3       3 137  410 0.200  126   390    0.920     0.951    0.464
4       4 137  547 0.267  111   501    0.810     0.916    0.596
5       5 137  684 0.333  108   609    0.788     0.890    0.725
6       6 136  820 0.400   82   691    0.603     0.843    0.823
7       7 137  957 0.467   64   755    0.467     0.789    0.899
8       8 137 1094 0.533   50   805    0.365     0.736    0.958
9       9 137 1231 0.600   20   825    0.146     0.670    0.982
10     10 136 1367 0.667    9   834    0.066     0.610    0.993
11     11 137 1504 0.733    3   837    0.022     0.557    0.996
12     12 137 1641 0.800    1   838    0.007     0.511    0.998
13     13 137 1778 0.867    2   840    0.015     0.472    1.000
14     14 136 1914 0.933    0   840    0.000     0.439    1.000
15     15 137 2051 1.000    0   840    0.000     0.410    1.000
```

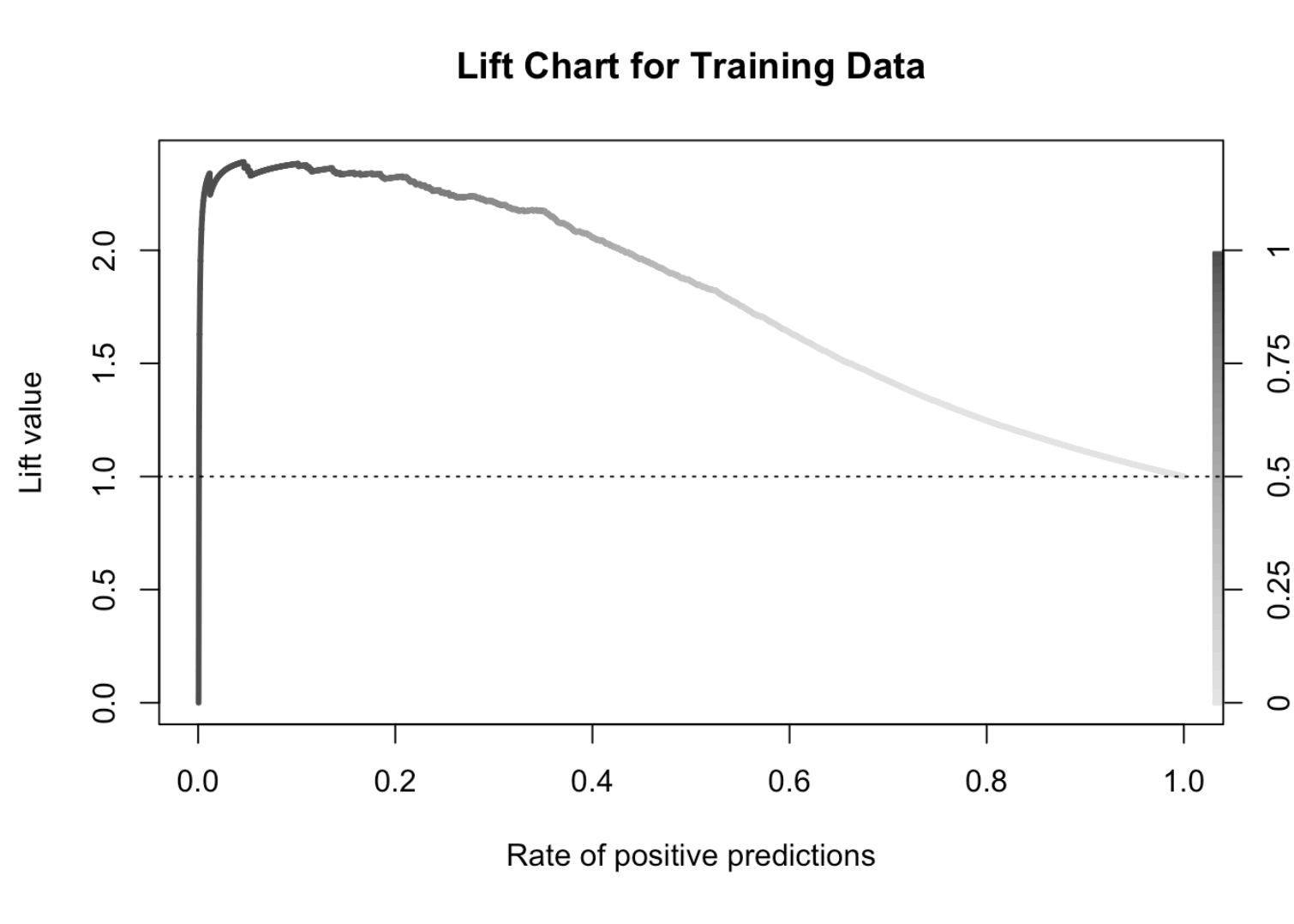
Let's examine the output above. In the first table with the data split into 10 buckets, let's examine the first row. Here we have 205 observations (1/10 of our data, or a depth of 0.1). Remember, these observations have been ranked by predicted probability so these observations have the highest probability of being a 1 according to our model. In these 205 observations, 200 of them had the response (target value of 1) which is a response rate of 0.976. Our original data had a total response rate (proportion of 1's) of only 0.41. This means that we did 2.382 (=0.976 / 0.41) times better than random with our top 10% of customers. Another way to think about this would be that if we were to randomly pick 10% of our data, we would have only expected to see 84 responses (target values of 1). Our **best** 10% had 200 responses. Again, this ratio is a value of 2.38. The table continues this calculation for each of the buckets of 10% of our data.

The lift chart displays how good that bucket is alone, while the cumulative lift chart (more popular one) displays how good you are up to that point. The cumulative lift and lift charts are the first plot displayed. The second plot is the response rate and cumulative response rate plot. Each point divided by the horizontal line at 0.41 (population response rate) gives the lift value in the first chart. The last chart is the cumulative capture rate plot. This tells us how much of the target 1's you captured with your model. The diagonal line would be random, so the further above the line the better the model.

Another way to calculate this is the `performance` function in R. This can easily calculate and plot the lift chart for us using the `measure = "lift"` and `x.measure = "rpp"` options. This plots the lift vs. the rate of positive predictions.

▼ Code

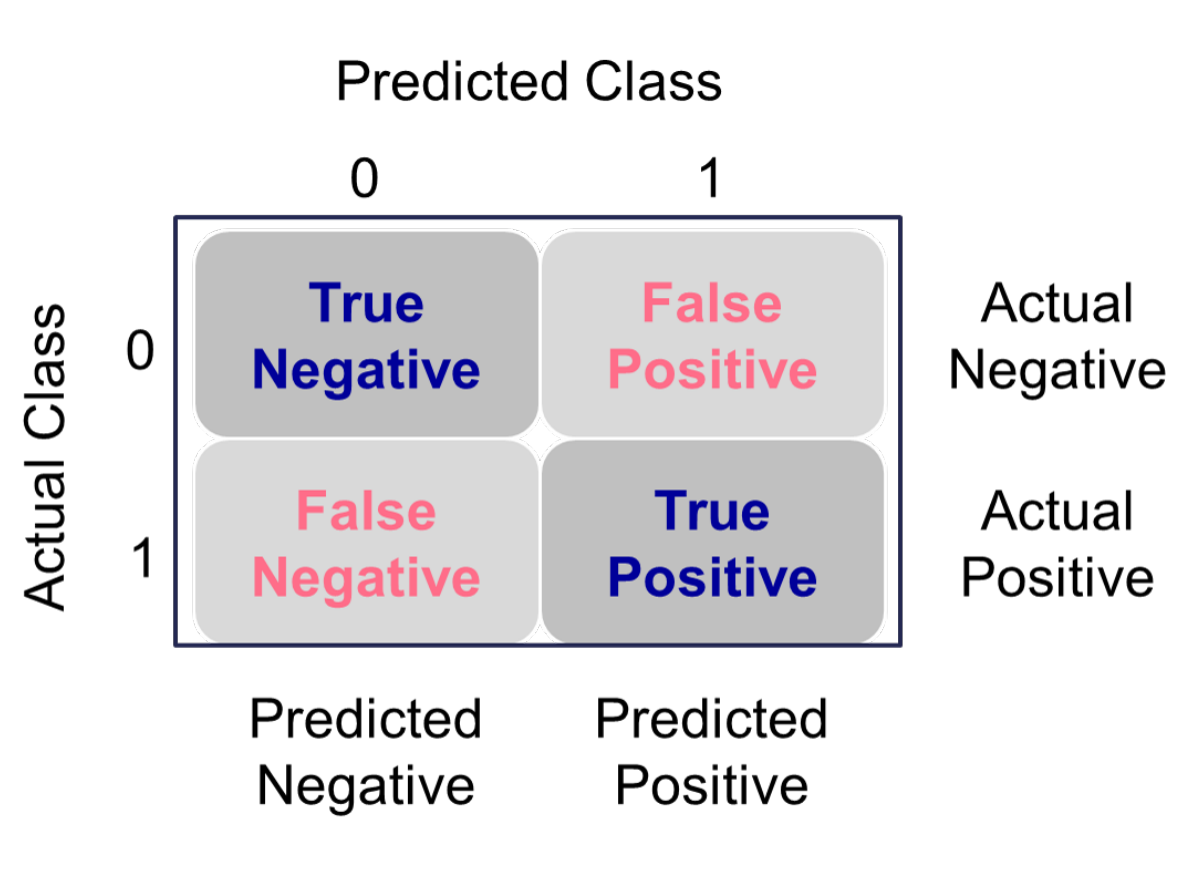
```
perf <- performance(pred, measure = "lift", x.measure = "rpp")
plot(perf, lwd = 3, colorize = TRUE, colorkey = TRUE,
     colorize.palette = rev(gray.colors(256)),
     main = "Lift Chart for Training Data")
abline(h = 1, lty = 3)
```



A common place to evaluate lift is at the population proportion. In our example above, the population proportion is approximately 0.41. At that point, we have a lift of approximately 2. In other words, if we were to pick the top 41% of homes identified by our model, we would be 2 times as likely to find a bonus eligible home as compared to randomly selecting from the population. This shows the value in our model in an interpretable way.

Accuracy & Error

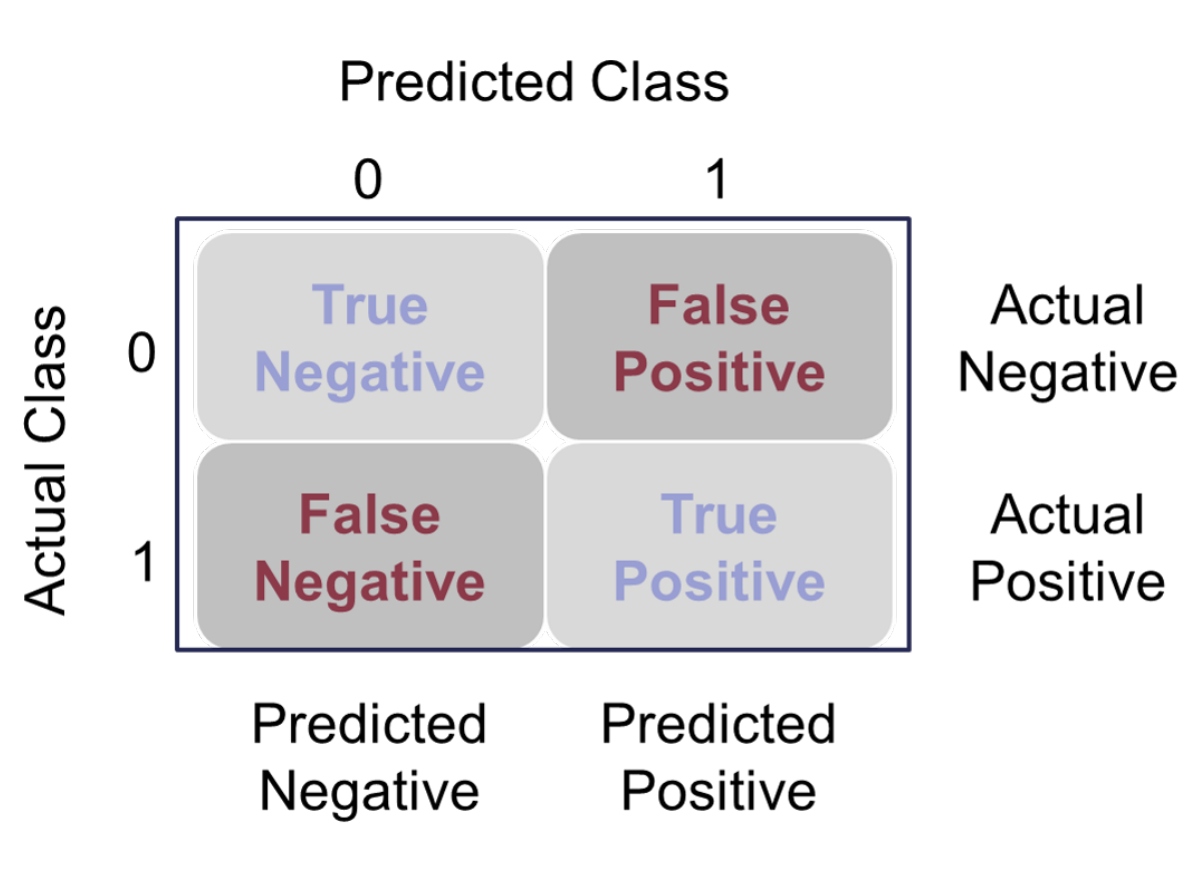
Accuracy and error rate are typically thought of when it comes to measuring the ability of a logistic regression model. Accuracy is essentially what percentage of events and non-events were predicted correctly.



$$Accuracy = \frac{TP + TN}{n}$$

Example Accuracy Calculation

The error would be the opposite of this.



$$Error = \frac{FP + FN}{n}$$

Example Error Calculation

However, caution should be used with these metrics as they can easily be fooled if *only* focusing on them. If your data has 10% events and 90% non-events, you can easily have a 90% accurate model by guessing non-events for every observation. Instead, having less accuracy might be better if we can predict both events and non-events. These numbers are still great to report! They are just not the best to decide which model is best.

Let's see how to do this in each of our softwares!

R Python SAS

R produces classification tables for any cut-off that you determine. Remember that a cut-off is simply the point where you decide to predict an event and non-event from your predicted probabilities.

We want to look at all classification tables for all values of cut-offs between 0 and 1. We can easily loop through this calculation with a `for` loop in R. However, the `measureit` function can do this for us. The inputs for this function are the predicted probabilities first, followed by the target variable. The `measure` option allows you to define additional measures to calculate at each cut-off. In the code below we ask for accuracy (`ACC`) and F1-score (`FSCR`). From there we combine these variables into a single dataframe and print the observations with the `print` function.

▼ Code

```
logit_meas <- measureit(train$p_hat, train$Bonus, measure = c(".
summary(logit_meas)
```

```
       Length Class  Mode   
Cutoff 2031   -none- numeric
Depth  2031   -none- numeric
TP     2031   -none- numeric
FP     2031   -none- numeric
TN     2031   -none- numeric
FN     2031   -none- numeric
ACC    2031   -none- numeric
FSCR   2031   -none- numeric
```

▼ Code

```
acc_table <- data.frame(Cutoff = logit_meas$Cutoff, Acc = logit_
head(arrange(acc_table, desc(Acc)), n = 10)
```

```
      Cutoff       Acc
1  0.5310044 0.8668942
2  0.5625426 0.8664066
3  0.5311089 0.8664066
4  0.5306445 0.8664066
5  0.5301028 0.8664066
6  0.5641566 0.8659191
7  0.5606633 0.8659191
8  0.5596935 0.8659191
9  0.5536701 0.8659191
10 0.5311367 0.8659191
```

From the output we can see the accuracy is maximized at 86.69%. The predicted probability that this occurs at (the optimal cut-off) is defined as 0.531. In other words, if our model predicts a probability above 0.531 then we should call this an event. Any predicted probability below 0.531 should be called a non-event, according to the accuracy metric.

There is more to model building than simply maximizing overall classification accuracy. These are good numbers to report, but not necessarily to choose models on.

Optimal Cut-off

Classification is a decision that is extraneous to statistical modeling. Although logistic regressions tend to work well in classification, it is a probability model and does not output events and non-events.

Classification assumes cost for each observation is the same, which is rarely true. It is always better to consider the costs of false positives and false negatives when considering cut-offs in classification. The previous sections talk about many ways to determine "optimal" cut-offs when costs are either not known or not necessary. However, if costs are known, they should drive the cut-off decision rather than modeling metrics that do not account for cost.